QCU ACADEMIC OS · AY 2026–2027

CC 102

Fundamentals of Programming — Complete Study Reference

Course: CC 102 | Units: 3 units | Language: Java · JDK 17 | Env: VS Code · Debian Linux

CHAPTER 00

Course Overview & How Java Actually Works

CC 102 teaches programming fundamentals through **Java** — one of the most widely used languages in enterprise software, Android development, and academic computer science education. Every concept here is language-agnostic at its core (variables, loops, functions exist in every language) but the exact syntax is Java-specific.

> **Critical Path — Twin Gate with WS 101**
>
> CC 102 (combined with CC 101) is required to unlock **CC 103** (Intermediate Programming) and **PT 101** (Platform Technologies) in Semester 2. CC 103 then unlocks **CC 104** (Data Structures & Algorithms), **CC 105** (Information Management), and **MS 101** (Discrete Math) in Year 2. Nearly your entire BSIT technical spine depends on truly understanding this course, not just passing it.

Where Java Came From

Java was created in 1991 by **James Gosling**, Mike Sheridan, and Patrick Naughton at Sun Microsystems, as part of what was internally called the **Green Project** — its first working product was a handheld remote control named **Star 7**. The language itself went through two names before "Java": first **Greentalk** (file extension `.gt`), then **Oak** — renamed because a

language called Oak already existed. The team settled on **Java**, after the Indonesian island where Java coffee originates. `JDK 1.0` officially released on January 23, 1996, and by 1995 Time magazine had already called it one of the Ten Best Products of the year.

Java's Core Features

Java technology is simultaneously a programming language, a development environment, an application environment, and a deployment environment. Twelve features define it:

Feature	What It Means
Object-oriented	Built around objects and classes — full treatment in Chapter 09 and again in PF 101
Simple	Designed to be easy to write, compile, debug, and learn
Secured	No explicit pointers; runs inside the JVM's managed sandbox
Platform Independent	Compiles to bytecode, not machine code — the same `.class` file runs on Windows, Linux, or macOS via each platform's own JVM
Robust	Strong memory management and exception handling reduce crash-prone code
Portable	Direct consequence of platform independence — write once, run anywhere
Architecture-neutral	No implementation-dependent features in the language spec itself
Dynamic	Adapts to an evolving environment; loads classes on demand
Interpreted	Bytecode is interpreted (and JIT-compiled) by the JVM at runtime
High-performance	Bytecode is close enough to native code that JIT compilation makes it fast
Multi-threaded	

Feature	What It Means
	Built-in support for running multiple threads of execution concurrently
Distributed	Designed with networked, distributed environments in mind

> **The One Feature That Matters Most Right Now**
>
> **Platform Independent** is the feature the next section is entirely about — it's the whole reason JDK/JRE/JVM exist as three separate layers instead of one.

Applet vs. Application

Java historically produces two kinds of programs:

Type	Definition	Status Today
Applet	A special application designed to run inside the context of a web browser	Deprecated — no modern browser runs Java applets anymore. You'll see it defined for exam purposes, but you will never build one.
Application	A stand-alone program that does not need a browser to run	This is what every program you write in CC 102 will be — including your `Hello.java` in Chapter 02

Java Program Structure

In the Java programming language, structure nests in exactly three levels:

```
A program is made up of one or more CLASSES
  A class contains one or more METHODS
    A method contains program STATEMENTS
```

A Java **application** always contains one method called `main` — the JVM loads the class and invokes `main()` to start execution. `main()` must always be declared `public`, `static`, and `void`. A single source file may define

more than one class, but at most ONE class in that file may be declared `public`, and that class name must exactly match the filename — the exact rule you'll rely on in Chapter 02.

JDK vs JRE vs JVM — The Three Letters You'll See Constantly

Term	Full Name	What It Actually Is
JVM	Java Virtual Machine	The engine that actually EXECUTES compiled Java bytecode. Different for each OS (Windows, Linux, macOS) — this is what makes Java portable.
JRE	Java Runtime Environment	JVM + core Java libraries. Enough to RUN Java programs, but not to write/compile new ones.
JDK	Java Development Kit	JRE + development tools (`javac` compiler, debugger, documentation tools). What you actually installed on your GT 30 Pro — JDK 17. Required to WRITE and compile code.

Compile Then Run — Java's Two-Step Process

Unlike Python (pure interpreter) or C (pure compiler to native machine code), Java does BOTH — this was covered conceptually in CC 101 Ch. 6, now you'll do it hands-on every single day.

1. **Write source code**
   You write human-readable code in a `.java` file, e.g. `Hello.java`
2. **Compile with javac**
   Terminal command: `javac Hello.java`. The compiler checks for syntax errors and converts your code into **bytecode** — a `Hello.class` file. This bytecode is NOT native machine code — it's platform-independent.

3 **Run with java**

Terminal command: `java Hello` (no `.class` extension). The JVM loads the bytecode and executes it, translating it to your specific machine's native instructions on the fly.

BASH — VS CODE TERMINAL

```
kian@debian:~/projects/CC102$ javac Hello.java
kian@debian:~/projects/CC102$ ls
Hello.class Hello.java
kian@debian:~/projects/CC102$ java Hello
Hello, World!
```

Exam & Practical Priority Map

Topic	Chapter	Weight	Difficulty
Logic Formulation (flowcharts, pseudocode)	01	High	Easy
Control Flow (if/else, switch)	05	Very High	Easy
Loops (for, while)	06	Very High	Medium
Variables & Data Types	03	High	Easy
Methods	07	High	Medium
Arrays	08	Medium	Medium
Operators	04	Medium	Easy
Intro to OOP (Classes/Objects)	09	Foundational	Medium

Logic Formulation — Algorithms, Pseudocode & Flowcharts

Before writing a single line of Java, you plan the LOGIC of a solution — independent of any programming language. This is CC 102 Weeks 2–3, and it's the skill exams test hardest in machine-problem items: can you trace a flowchart or pseudocode by hand and predict its output BEFORE touching a keyboard.

The Programming Cycle

Every program — from a one-line script to XoDos-Ark itself — follows the same four-stage cycle.

1. **Problem definition**
   State exactly what the program must accomplish — inputs, outputs, and constraints.
2. **Problem analysis**
   Break the problem into smaller, manageable parts before attempting a solution.
3. **Algorithm development**
   Design the step-by-step logic — usually as pseudocode or a flowchart — independent of syntax.
4. **Coding & debugging**
   Translate the algorithm into actual Java syntax, then test and fix errors.

> **Why This Comes Before Chapter 02**
> Steps 1–3 happen entirely on paper — no compiler involved. Jumping straight to coding (step 4) without planning is the #1 reason beginners get stuck staring at a blank file. Every conditional and loop you write starting Chapter 05 already has its logic fully worked out here first.

Algorithm

A step-by-step problem-solving procedure — an established, finite sequence of instructions for solving a problem.

Characteristic	Requirement
Precision	Each step or instruction must be specified exactly — no ambiguity
Finiteness	There must be a finite number of steps — it must eventually stop
Output	There must be a definite output — a purposeless algorithm proves nothing

Pseudocode

A generic way of describing an algorithm without using the syntax of any specific programming language. It can't be executed on a real computer, but it's written at roughly the same level of detail as real code, and reads like structured, plain English.

PSEUDOCODE

```
If student's grade is greater than or equal to 60
    Print "passed"
else
    Print "failed"
```

This Is Literally Chapter 05, Just Without Syntax

Compare this to the `if / else` block you'll write in Chapter 05 — same logic, same structure, just missing curly braces, parentheses, and semicolons. Get comfortable reasoning in pseudocode first; the Java syntax is the easy part once the logic is right.

Flowcharts

A flowchart is a chart of symbols representing computer operations, describing how a program performs. Symbols are connected by flow lines (straight lines with arrows) that show the direction of data or control from one point to another.

Symbol	Shape	Meaning
Terminal block	Oval / rounded rectangle	Marks the **Start** or **End**. A start has no incoming flow line and exactly one outgoing; an end has one incoming and none outgoing.
Initialization	Hexagon	Declares and/or initializes the variables needed for the process (e.g. `X = 10`)
Input / Output	Parallelogram	Getting input or displaying output — one entrance, one exit (e.g. `Get X`, `Display X`)
Process	Rectangle	A processing step — calculations, assignments, opening/closing files. One entrance, one exit.
Decision	Diamond	Tests a condition. One entrance, exactly **two** exits — one TRUE, one FALSE.
Connector	Small circle	Links two non-adjacent sections of a flowchart. Use sparingly — overuse clutters readability.
Flow lines	Arrows	Show the direction of data/ control flow between symbols.

> **Decision Boxes Must Be Mathematical, Not Essay-Form**
>
> A condition inside a diamond must resolve to TRUE or FALSE, e.g. `X < 20?` — NOT "Is variable X less than 20?" Essay-form conditions aren't valid flowchart logic.

Single vs. Dual Alternative Selection

A decision box doesn't always have to do something on both paths.

Structure	Behavior	Example
Single alternative	Only the TRUE (or only the FALSE) path performs an action; the other just merges through	if x < y, print x
Dual alternative	Both TRUE and FALSE paths perform a (different) action	if x < y, print x, otherwise print y

Multiple Decision Boxes

When more than two outcomes are possible, chain decision boxes together. Worked example: ask the user to enter a number 1–3. If 1, print "Hello"; if 2, print "Hi"; if 3, print "Bye"; otherwise print "Invalid."

FLOWCHART LOGIC — TRACED TOP TO BOTTOM

```
Start
  x = 0
  Get x
  x == 1?  --Yes--> "Hello" --> End
     |No
  x == 2?  --Yes--> "Hi"    --> End
     |No
  x == 3?  --Yes--> "Bye"   --> End
     |No
  "Invalid" --> End
```

Decision boxes can also test **ranges** instead of exact matches — chain them from lowest to highest (or highest to lowest) value:

Department Number	Supervisor
1–3	Mr. X
4–7	Mr. Y
8–9	Mr. Z

FLOWCHART LOGIC

```
Dept <= 3?  --Yes--> "Mr. X"
     |No
Dept <= 7?  --Yes--> "Mr. Y"
     |No
            "Mr. Z"
```

Relational Operators

Compare two values and return a Boolean value depending on whether the test condition is true or false.

Operator	Meaning
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to
`==`	Is equal to
`!=` or `<>`	Not equal to

> **Preview — Chapter 04**
> These exact symbols (except `<>`, which Java doesn't use) carry directly into Java's relational operators. The one that trips people up most: `==` for comparison vs. `=` for assignment — a bug you'll meet again in Chapter 04.

Logical Operators

Conditions can be combined using three basic logical operators: **AND**, **OR**, and **NOT**.

Condition A	Condition B	A AND B	A OR B
F	F	F	F
F	T	F	T

Condition A	Condition B	A AND B	A OR B
T	F	F	T
T	T	T	T

NOT simply reverses a condition's result: `!TRUE` → FALSE, and `!FALSE` → TRUE.

Preview — Chapter 04

AND, OR, and NOT become Java's `&&`, `||`, and `!` operators. The truth tables above are exactly the ones you'll use to trace compound `if` conditions later.

Looping Constructs (Repetition Structures)

A repetition structure — also called a loop or iteration — repeats one or more instructions until a certain condition is met. This is a **pre-test repetition structure**: the condition is tested BEFORE any action runs, so if the condition is false from the start, the loop body never executes at all.

The #1 Flowchart Bug — the Infinite Loop

The action inside a loop MUST eventually change the value being tested, or the loop never terminates. `X < Y?` → Display X → back to `X < Y?` with X never updated is an infinite loop. The fix: add `X = X + 1` inside the loop body so the condition eventually becomes false.

CORRECTED LOOP LOGIC

```
X < Y?  --Yes--> Display X --> X = X + 1 --> (back to X < Y?)
   |No
  (exit loop)
```

Multiple / Nested Loops

A loop can exist within another loop. When a flowchart has multiple loops, it's critical to identify which condition terminates the WHOLE flowchart, not

just the inner one — the same rule you'll apply to nested `for` loops in Chapter 06.

Full Worked Example

Create a flowchart that asks the user to enter a gender. If 'M', display "MALE"; if 'F', display "FEMALE"; otherwise display "Invalid Gender" and let the user try again. Repeat the whole process 5 times.

FLOWCHART LOGIC — TRACED TOP TO BOTTOM

```
Start
  Gen = "", Ctr = 1
  [loop top] Ctr <= 5?  --No--> Stop
     |Yes
  Get Gen
  Gen != 'M' AND Gen != 'F'?  --Yes--> "Invalid" --> Ctr = Ctr + 1 -->
back to [loop top]
     |No
  Gen == 'M'?  --Yes--> "Male"   --> Ctr = Ctr + 1 --> back to [loop top]
     |No
             "Female" --> Ctr = Ctr + 1 --> back to [loop top]
```

The counter `Ctr` is what bounds this loop to exactly 5 repetitions — initialized once before the loop, tested at the top (pre-test), and incremented once per pass no matter which branch fired. This exact counter pattern is what becomes a Java `for` loop in Chapter 06.

CHAPTER 02

Setup & Your First Program

Every Java program, no matter how large, starts from the exact same anatomical structure. This chapter dissects that structure piece by piece so nothing is ever "magic syntax" you just memorize.

Anatomy of a Java Program

HELLO.JAVA

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Piece	Meaning
`public`	Access modifier — this class is visible to any other class/file that needs it. Covered fully in Ch. 09.
`class Hello`	Declares a class named `Hello`. **CRITICAL RULE:** the filename MUST exactly match the public class name — `Hello.java` for `class Hello`. Mismatch = compile error.
`public static void main(String[] args)`	The **entry point** — the JVM looks for exactly this method signature to know where to start executing. `static` means it belongs to the class itself, not an object instance. `void` means it returns nothing. `String[] args` holds command-line arguments passed at runtime.
`System.out.println(...)`	`System` is a built-in class, `out` is its output stream object, `println` prints text then moves to a new line. `print` (no "ln") does NOT add a new line.
`;`	Every statement in Java MUST end with a semicolon. Forgetting this is the #1 beginner compile error.
`{ }`	Curly braces define a "block" — the class body, the method body. Every opening brace needs a matching closing brace.

EXPECTED OUTPUT

```
Hello, World!
```

Compiler Errors You WILL Hit — And What They Mean

Error Message	What Actually Went Wrong	Fix
`error: ';' expected`	Missing semicolon at end of a statement	Add the `;` at the indicated line
`error: class Hello is public, should be declared in a file named Hello.java`	Filename doesn't match the public class name	Rename the file OR rename the class to match
`error: cannot find symbol`	Using a variable/method name that doesn't exist — often a typo or wrong capitalization (Java is case-sensitive)	Check exact spelling and capitalization
`error: reached end of file while parsing`	Missing a closing curly brace `}` somewhere	Count your opening vs closing braces
`Error: Could not find or load main class`	Trying to `java` run a class with no `main` method, or running the wrong filename	Confirm the `main` method exists and matches the class you're running

Comments

JAVA

```java
// Single-line comment — everything after // is ignored

/* Multi-line comment
   spans several lines
   until closed */

/**
 * Javadoc comment — generates official documentation
```

```
 * @param args command-line arguments
 */
```

Your Debian/VS Code Workflow

1. **Open your project folder**
   VS Code → `/home/kian/projects/CC102`
2. **Create a new .java file**
   Match filename to class name exactly, e.g. `Hello.java`
3. **Write your code, save (Ctrl+S)**
   The Extension Pack for Java gives you autocomplete and inline error squiggles
4. **Open integrated terminal**
   `Ctrl+\`` — then `javac Hello.java` followed by `java Hello`

CHAPTER 03

Variables & Data Types

A variable is a named container that holds a value in memory. Java is **statically typed** — you must declare exactly what TYPE of data a variable will hold, and it can never change type afterward. This is different from Python/JavaScript, which are dynamically typed.

Variable Declaration Syntax

JAVA

```java
dataType variableName = value;

int age = 18;
double gpa = 1.75;
String name = "Kian";
boolean isEnrolled = true;
```

Types of Variables — Local, Instance, and Static

Where a variable is declared inside a class determines its **scope** (where it can be used) and its **lifetime** (how long it stays in memory). Java recognizes three kinds:

Type	Declared Where	Lives For
Local variable	Inside a method or constructor body	Only while that method is running — destroyed the moment it returns. Every variable you've written inside `main()` so far has been local.
Instance variable	Inside the class, but outside any method	As long as the object exists — each object (instance) gets its OWN separate copy
Static variable	Inside the class, outside any method, marked `static`	The entire lifetime of the program — shared by ALL objects of that class, not copied per object. A static variable can never be declared local.

VARIABLETYPES.JAVA

```
class A {
    int data = 50;       // instance variable
    static int m = 100;  // static variable

    void method() {
        int n = 90;        // local variable
    }
}
```

Forward Link — Chapter 09 (Intro to OOP)

Instance and static variables won't fully click until you meet classes and objects properly in Chapter 09. For now: local = inside a method (temporary), instance = inside the class, one copy per object, static = inside the class, one copy shared by every object — this is also exactly why `main()` itself is marked `static` back in Chapter 02, since the JVM calls it before any object exists.

The 8 Primitive Data Types

Primitives store raw values directly in memory (not references to objects) — the most memory-efficient data types in Java.

Type	Size	Range / Values	Example
`byte`	1 byte (8 bits)	-128 to 127	`byte age = 18;`
`short`	2 bytes (16 bits)	-32,768 to 32,767	`short year = 2026;`
`int`	4 bytes (32 bits)	≈ -2.1 billion to 2.1 billion	`int score = 95;` — most commonly used integer type
`long`	8 bytes (64 bits)	≈ ±9.2 quintillion	`long population = 114000000L;` — note the `L` suffix
`float`	4 bytes	~6-7 decimal digits precision	`float price = 19.99f;` — note the `f` suffix
`double`	8 bytes	~15 decimal digits precision	`double gpa = 1.75;` — default for decimals, most commonly used
`char`	2 bytes	A single Unicode character	`char grade = 'A';` — single quotes, exactly one character

Type	Size	Range / Values	Example
`boolean`	1 bit (conceptually)	`true` or `false` only	`boolean isPassed = true;`

> **Forward Link — CC 101 Ch. 4 Data Representation**
>
> Notice the byte sizes above match EXACTLY what you learned in CC 101's Data Representation chapter. An `int` being 4 bytes (32 bits) is not arbitrary — it directly determines its range: 2^{32} possible values, split between negative and positive (signed representation). This is why `byte` maxes at 127, not 256 — half the range is reserved for negative numbers.

String — Not a Primitive!

`String` is technically a **class**, not a primitive — it's capitalized because it's an object type. Strings use double quotes; chars use single quotes. This is a very common beginner mix-up.

JAVA

```
String course = "CC 102";  // double quotes — text
char grade = 'A';           // single quotes — ONE character only
```

Type Casting

Converting a value from one data type to another.

Type	Direction	Syntax	Risk
Widening (Implicit)	Small type → Large type	Automatic, no cast needed: `int → long → float → double`	Safe — no data loss
Narrowing (Explicit)	Large type → Small type	Must manually cast: `double d = 9.7; int i = (int) d;`	Can lose data (decimal truncated, or overflow if the value is too big)

CASTING.JAVA

```
public class Casting {
    public static void
main(String[] args) {
        double gpa = 1.75;
        int rounded = (int) gpa;

System.out.println(rounded);

        int wholeNum = 10;
        double asDecimal =
wholeNum;

System.out.println(asDecimal);
    }
}
```

EXPECTED OUTPUT

```
1
10.0
```

LINE EXPANSION — EXECUTION TRACE

Step	Line	Variable State	Notes
1	`double gpa = 1.75;`	gpa = 1.75	Declares and initializes a double
2	`int rounded = (int) gpa;`	gpa = 1.75, rounded = 1	Narrowing cast TRUNCATES (chops off) the decimal — does NOT round. 1.75 becomes 1, not 2.
3	`System.out.println(rounded);`	—	Prints: 1
4	`int wholeNum = 10;`	wholeNum = 10	Declares an int
5			Widening — automatic, no

Step	Line	Variable State	Notes
	`double asDecimal = wholeNum;`	`wholeNum = 10, asDecimal = 10.0`	cast syntax needed
6	`System.out.println(asDecimal);`	—	Prints: 10.0 (note the .0 — it's genuinely a double now)

Gotcha Alert — Casting Truncates, It Does NOT Round

`(int) 1.75` gives you `1`, not `2`. Casting simply chops off everything after the decimal point. To actually ROUND, use `Math.round(1.75)` instead, which correctly gives `2`.

Identifiers, Naming Conventions & Reserved Keywords

An **identifier** is any name you choose for a class, variable, or method. Every identifier must start with a letter (A–Z, a–z), a dollar sign (`$`), or an underscore (`_`) — after that first character, any combination of letters, digits, `$`, or `_` is allowed. Identifiers are case-sensitive, so `score` and `Score` are two different names. **Keywords** (also called reserved words — things like `class`, `public`, `static`, `int`, `if`) can never be used as an identifier, since Java itself already uses them as part of its syntax.

Rule	Example
Must start with a letter, `$`, or `_`	Legal: `age`, `$salary`, `_value`, `__1_value`
Cannot start with a digit, or any other symbol	Illegal: `123abc`, `-salary`
Variables use **camelCase**	`studentName`, `totalScore` — not `student_name`
Classes use **PascalCase**	`StudentRecord`, `Hello`
Constants use **UPPER_SNAKE_CASE**	`final int MAX_SCORE = 100;`

Rule	Example
Cannot use reserved keywords	`class, public, static, void, int, if, for` — these are reserved by Java itself

Constants — the final Keyword

JAVA

```
final double PI = 3.14159;
final String UNIVERSITY = "QCU";
// PI = 3.14; ← this line would cause a compile error — final means unchangeable
```

CHAPTER 04

Operators

Operators perform operations on variables and values. Java groups them into four families you'll use constantly: arithmetic, relational, logical, and assignment.

Arithmetic Operators

Operator	Name	Example	Result
`+`	Addition	`5 + 3`	8
`-`	Subtraction	`5 - 3`	2
`*`	Multiplication	`5 * 3`	15
`/`	Division	`7 / 2`	3 (int division TRUNCATES!)
`%`	Modulus (remainder)	`7 % 2`	1

Operator	Name	Example	Result
`++`	Increment	`x++`	Adds 1 to x
`--`	Decrement	`x--`	Subtracts 1 from x

Gotcha Alert — Integer Division Truncates

`7 / 2` with two `int` operands gives `3`, NOT `3.5`. Java performs integer division when both operands are integers — it simply discards the remainder. To get a decimal result, at least ONE operand must be a double: `7.0 / 2` gives `3.5`. This trips up nearly every beginner at least once.

Pre vs Post Increment — A Classic Trap

INCREMENT.JAVA

```
public class Increment {
    public static void
main(String[] args) {
        int a = 5;
        System.out.println(a++);
        System.out.println(a);

        int b = 5;
        System.out.println(++b);
        System.out.println(b);
    }
}
```

EXPECTED OUTPUT

```
5
6
6
6
```

LINE EXPANSION – EXECUTION TRACE

Step	Expression	Behavior
1	`a++` (post-increment)	Uses the CURRENT value (5) first, THEN increments. Prints 5, a becomes 6.
2	`a`	

Step	Expression	Behavior
		Now 6, confirming the increment already happened.
3	`++b` (pre-increment)	Increments FIRST, THEN uses the new value. b becomes 6, prints 6 immediately.
4	`b`	Still 6 — same result as step 3, just confirming.

Relational (Comparison) Operators

Always evaluate to a `boolean` (`true` / `false`) — the backbone of every condition and loop.

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	true
`!=`	Not equal to	`5 != 3`	true
`>`	Greater than	`5 > 3`	true
`<`	Less than	`5 < 3`	false
`>=` ▲	Greater than or equal	`5 >= 5`	true
`<=`	Less than or equal	`3 <= 5`	true

Gotcha Alert — = vs ==

A SINGLE `=` is the ASSIGNMENT operator (stores a value). A DOUBLE `==` is the COMPARISON operator (checks equality). Writing `if (x = 5)` instead of `if (x == 5)` is a classic beginner bug — in Java this actually causes a compile error for boolean contexts (unlike C), which is one small mercy, but the confusion still costs time.

Logical Operators

Operator	Name	Rule	Example
`&&`	Logical AND	true only if BOTH sides are true	`(age >= 18) && (isEnrolled)`
`||`	Logical OR	true if AT LEAST ONE side is true	`(isRegular) || (isIrregular)`
`^`	Logical XOR (Exclusive OR)	true only if the two sides are DIFFERENT (one true, one false)	`true ^ false` → true, but `true ^ true` → false
`!`	Logical NOT	Flips true↔false	`!isPassed` (true if NOT passed)

Assignment Operators

Operator	Equivalent To	Example
`=`	Basic assignment	`x = 5;`
`+=`	`x = x + value`	`x += 3;` (x increases by 3)
`-=`	`x = x - value`	`x -= 2;`
`*=`	`x = x * value`	`x *= 2;`
`/=`	`x = x / value`	`x /= 2;`
`%=`	`x = x % value`	`x %= 3;`

Operator Precedence — Order of Operations

Just like in math (PEMDAS), Java evaluates expressions in a strict order when no parentheses are given. When in doubt, USE parentheses — it costs nothing and removes ambiguity entirely.

Priority	Operators
1 (highest)	`( )` parentheses, `++` `--`
2	`*` `/` `%`
3	`+` `-`
4	`< > <= >=`
5	`== !=`
6	`&&`
7 (lowest)	`||`

Let's Try — Precedence Practice

Walk through these the same way your professor does: resolve `* / %` left-to-right first, then `+ -` left-to-right, respecting any parentheses.

EXAMPLE 1 — `7 - 24 / 8 X 4 + 6`

Step	Expression	Why
1	`7 - 3 x 4 + 6`	`24 / 8` resolves first (division, left-to-right before multiplication of equal priority)
2	`7 - 12 + 6`	`3 x 4` resolves next

EXAMPLE 2 — `(17 - 6 / 2) + 4 X 3`

Step	Expression	Why
1	`(17 - 3) + 4 x 3`	Inside the parentheses, `6 / 2` resolves first
2	`(14) + 4 x 3`	`17 - 3` finishes the parentheses group
3	`14 + 12`	`4 x 3` resolves

Step	Expression	Why
3	`-5 + 6`	`7 - 12` resolves (addition / subtraction now left-to-right)
4	= 1	Final answer

Step	Expression	Why
		(higher priority than the remaining `+`)
4	= 26	Final answer

CHAPTER 05

Control Flow — Conditionals

Conditionals let a program make DECISIONS — executing different code depending on whether an expression is true or false. This is one of the highest-weighted topics in CC 102 practicals.

if / else if / else

GRADE.JAVA

```
public class Grade {
    public static void
main(String[] args) {
        int score = 85;

        if (score >= 90) {

System.out.println("Excellent");
        } else if (score >= 75) {
```

EXPECTED OUTPUT

```
Passed
```

```
System.out.println("Passed");
        } else {

System.out.println("Failed");
        }
    }
}
```

LINE EXPANSION — EXECUTION TRACE

Step	Check	Result	Action
1	`score >= 90`	85 >= 90 → false	Skip this block, check next condition
2	`score >= 75`	85 >= 75 → true	Execute this block: print "Passed". Once a branch matches, ALL remaining else-if/else branches are skipped entirely.
3	(else branch)	never reached	Program exits the if-chain

Key Rule — Only ONE Branch Ever Executes

In an if/else-if/else chain, Java checks conditions TOP TO BOTTOM and stops at the FIRST true one. Even if a later condition would also technically be true, it's never checked. Order your conditions carefully — this is why grade brackets are always checked from HIGHEST to LOWEST.

switch Statement

An alternative to long if/else-if chains when checking ONE variable against many possible exact values.

DAYTYPE.JAVA

```
public class DayType {
    public static void
main(String[] args) {
        int day = 3;
        String result;

        switch (day) {
            case 1:
                result =
"Monday";
                break;
            case 2:
                result =
"Tuesday";
                break;
            case 3:
                result =
"Wednesday";
                break;
            default:
                result =
"Invalid";
        }

System.out.println(result);
    }
}
```

EXPECTED OUTPUT

```
Wednesday
```

Gotcha Alert — Forgetting break Causes "Fall-Through"

Without `break;`, execution CONTINUES into the next case block regardless of whether it matches. If the example above were missing `break;` after case 3, it would print "Wednesday" AND fall through to execute `default` too, printing "Invalid" right after. Always include `break;` unless you deliberately want fall-through behavior (rare, but occasionally intentional for grouping cases).

Ternary Operator — Compact if/else

A shorthand for simple if/else assignments, using `?` and `:`.

JAVA

```java
int age = 18;
String status = (age >= 18) ? "Adult" : "Minor";
// Reads as: condition ? valueIfTrue : valueIfFalse
System.out.println(status);  // Adult
```

Nested Conditionals

An if statement placed INSIDE another if statement — used when a decision depends on multiple layered conditions.

JAVA

```java
int gpa = 2;
boolean hasBacksubject = false;

if (gpa <= 3) {
    if (!hasBacksubject) {
        System.out.println("Eligible for Dean's List");
    } else {
        System.out.println("GPA qualifies, but has a back subject");
    }
} else {
    System.out.println("Not eligible");
}
// Output: Eligible for Dean's List
```

Filipino Analogy — Conditionals

Think of a **jeepney fare checker**: "IF you're a student with valid ID, fare is discounted. ELSE IF you're a senior citizen, also discounted. ELSE, regular fare applies." The driver checks conditions in order and applies the FIRST one that matches — exactly like an if/else-if/else chain.

Common Compiler/Logic Errors in Conditionals

Mistake	Why It's Wrong
`if (x = 5)`	Assignment instead of comparison — should be `==`
Missing `break;` in switch	Causes unintended fall-through to next case
Checking ranges out of order	e.g. checking `score >= 75` before `score >= 90` — the 90+ scores get caught by the first (wrong) branch
Comparing Strings with `==`	Should use `.equals()` instead — `==` compares object references, not actual text content, for non-primitive types

CHAPTER 06

Loops

Loops repeat a block of code multiple times without rewriting it. This is where beginners most commonly get stuck on infinite loops and off-by-one errors — the execution traces below are built specifically to make the invisible "how many times does this actually run" visible.

for Loop — When You Know the Number of Repetitions

JAVA

```java
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Part	Runs When	In This Example
Initialization	ONCE, before the loop starts	`int i = 0;` — creates the counter
Condition	Checked BEFORE every single iteration	`i < 5` — loop continues while true
Update	AFTER every iteration completes	`i++` — increments the counter

FORDEMO.JAVA

```
public class ForDemo {
    public static void
main(String[] args) {
        for (int i = 0; i < 5; i+
+) {

System.out.println(i);
        }
    }
}
```

EXPECTED OUTPUT

```
0
1
2
3
4
```

LINE EXPANSION — EXECUTION TRACE

Iteration	Check i < 5	Prints	After: i++
Start	i = 0 (initialized)	—	—
1	0 < 5 → true	0	i becomes 1
2	1 < 5 → true	1	i becomes 2
3	2 < 5 → true	2	i becomes 3
4	3 < 5 → true	3	i becomes 4
5	4 < 5 → true	4	i becomes 5
6	5 < 5 → **false**	—	

Iteration	Check i < 5	Prints	After: i++
▲			Loop STOPS. Body never runs a 6th time.

Gotcha Alert — Off-By-One Errors

Notice the loop prints 0,1,2,3,4 — FIVE numbers, even though the condition says "less than 5," not "less than or equal to 5." This is the #1 source of off-by-one bugs. If you wanted 1 through 5 instead, you'd write `for (int i = 1; i <= 5; i++)`. Always trace through manually the first few times until this becomes automatic.

while Loop — When You Don't Know the Exact Count

Checks the condition BEFORE each iteration — same as `for`, but the counter/condition logic is entirely manual.

WHILEDEMO.JAVA

```
public class WhileDemo {
    public static void
main(String[] args) {
        int count = 0;
        while (count < 3) {

System.out.println("Count: " +
count);
            count++;
        }
    }
}
```

EXPECTED OUTPUT

```
Count: 0
Count: 1
Count: 2
```

Gotcha Alert — The Infinite Loop Trap

If you forget `count++;` inside the loop body, `count` NEVER changes, the condition `count < 3` stays true FOREVER, and your program hangs

indefinitely (you'll need Ctrl+C in the terminal to force-stop it). ALWAYS verify your loop has a way to eventually make the condition false.

do-while Loop — Guaranteed At Least One Execution

Checks the condition AFTER running the body — meaning the body ALWAYS runs at least once, even if the condition is false from the start.

JAVA

```
int num = 10;
do {
    System.out.println("Runs once: " + num);
} while (num < 5);
// Output: "Runs once: 10" — prints ONE time even though 10 < 5 is false
```

for vs while vs do-while — When to Use Each

Loop	Checks Condition	Best When
`for`	Before each iteration	You know the exact number of repetitions in advance (e.g. "loop 10 times")
`while`	Before each iteration	Repetitions depend on a changing condition, unknown count (e.g. "keep asking until valid input")
`do-while`	After each iteration	The body must run AT LEAST once regardless of the condition (e.g. menu systems: show the menu once before checking if the user wants to exit)

break and continue

Keyword	Effect
`break;`	

Keyword	Effect
	Immediately EXITS the loop entirely, skipping all remaining iterations
`continue;`	Skips the REST of the current iteration only, then moves to the next one — the loop keeps going

JAVA

```java
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    if (i == 5) break;
    System.out.println(i);
}
```

EXPECTED OUTPUT

```
1
2
4
```

LINE EXPANSION — EXECUTION TRACE

i value	i == 3?	i == 5?	Result
1	no	no	Prints 1
2	no	no	Prints 2
3	**yes**	—	`continue` — skips print, jumps to i++ immediately
4	no	no	Prints 4
5	no	**yes**	`break` — loop ends immediately, 5 is never printed

Nested Loops

A loop inside another loop — the inner loop completes ALL its iterations for EACH single iteration of the outer loop.

JAVA

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++)
{
        System.out.println(i +
"-" + j);
    }
}
```

EXPECTED OUTPUT

```
1-1
1-2
1-3
2-1
2-2
2-3
```

Total iterations = outer × inner = 2 × 3 = **6 total prints**. This is the exact mental model you'll need for 2D array traversal in the next chapter.

Filipino Analogy — Nested Loops

Think of a **Jollibee branch audit**: outer loop = for each branch (Branch 1, Branch 2), inner loop = for each register at that branch (Register 1, 2, 3). You fully audit ALL registers at Branch 1 before moving to Branch 2 — never jumping back and forth. That's exactly how nested loops execute.

CHAPTER 07

Methods

A method is a named, reusable block of code that performs a specific task. Methods let you write logic ONCE and call it as many times as needed — the foundation of organized, non-repetitive programming, and the direct stepping stone to Object-Oriented Programming.

Method Anatomy

JAVA

```
accessModifier returnType methodName(parameterType paramName) {
    // method body
    return value; // only if returnType is not void
}

public static int addNumbers(int a, int b) {
    int sum = a + b;
    return sum;
}
```

Part	Meaning
`public`	Access modifier — who can call this method (covered fully in Ch. 09)
`static`	Belongs to the class itself, not to an object instance — callable directly without creating an object first
`int` (return type)	The data type this method WILL send back. Use `void` if it returns nothing.
`addNumbers`	The method's name — called using this name plus parentheses
`(int a, int b)`	**Parameters** — the inputs the method expects, with their types
`return sum;`	Sends the value back to wherever the method was called. Immediately EXITS the method — any code after `return` in that path never runs.

Calling a Method

CALCULATOR.JAVA

```
public class Calculator {
```

EXPECTED OUTPUT

```
8
```

```
    public static int
addNumbers(int a, int b) {
        int sum = a + b;
        return sum;
    }

    public static void
main(String[] args) {
        int result = addNumbers(5,
3);

System.out.println(result);

System.out.println(addNumbers(10,
20));
    }
}
```

```
30
```

LINE EXPANSION — EXECUTION TRACE

Step	What Happens
1	`main` calls `addNumbers(5, 3)` — execution JUMPS out of main into the addNumbers method
2	Inside addNumbers: parameter `a = 5`, `b = 3` (copies of the values passed in)
3	`int sum = a + b;` → sum = 8
4	`return sum;` → sends 8 back, execution JUMPS BACK to where it was called in main
5	`result` now holds 8. `println(result)` prints 8.
6	Second call: `addNumbers(10, 20)` runs independently — new a=10, b=20, returns 30, printed directly

Step	What Happens
	without storing in a variable first.

Parameters vs Arguments — Don't Confuse These

Term	Definition	In the Example Above
Parameter	The variable name listed in the method's DEFINITION	`a` and `b` in `addNumbers(int a, int b)`
Argument	The actual VALUE passed in when CALLING the method	`5` and `3` in `addNumbers(5, 3)`

Method Overloading

Java allows MULTIPLE methods with the SAME NAME, as long as their parameter lists differ (different number or types of parameters). Java determines which one to call based on what you pass in.

JAVA

```java
public static int add(int a, int b) {
    return a + b;
}

public static double add(double a, double b) {
    return a + b;
}

public static int add(int a, int b, int c) {
    return a + b + c;
}

// add(2, 3)        -> calls the first version (int, int)
// add(2.5, 3.5)     -> calls the second version (double, double)
// add(1, 2, 3)      -> calls the third version (three ints)
```

void vs Return Types

Type	Behavior
`void`	Method performs an action but sends NOTHING back. Cannot be used in an expression like `int x = someVoidMethod();` — that's a compile error.
Any type (`int` , `String` , etc.)	Method MUST use `return` to send back a value of that exact type on every possible code path.

Scope — Where a Variable "Exists"

Scope is the region of code where a variable is accessible. A variable declared INSIDE a method only exists within that method — it cannot be accessed from outside.

JAVA

```
public static void methodA() {
    int x = 10;  // x only exists inside methodA
}

public static void methodB() {
    System.out.println(x); // ERROR — x doesn't exist here!
}
```

Gotcha Alert — Variables Are Local by Default

Each method has its OWN isolated set of variables. Parameters and local variables declared inside a method cease to exist the moment that method finishes executing (`return` s or reaches its closing brace). This is called the variable going "out of scope."

Forward Link — PF 101 (Year 2)

Method overloading here is your FIRST taste of **polymorphism** — one of the four pillars of Object-Oriented Programming you'll formally study in **PF 101** (Object-Oriented Programming, Year 2 Sem 1). The core idea — "same name, different behavior based on context" — scales up directly into method overriding with inheritance.

CHAPTER 08

Arrays

An array is a fixed-size container that holds MULTIPLE values of the SAME data type, stored in contiguous memory and accessed by numeric index. Arrays are the foundation for the far more advanced data structures you'll study in **CC 104** (Data Structures & Algorithms, Year 2).

Declaring & Initializing Arrays

JAVA

```java
// Method 1: Declare then assign each index
int[] scores = new int[5];
scores[0] = 85;
scores[1] = 90;

// Method 2: Declare and initialize together (array literal)
int[] grades = {85, 90, 78, 92, 88};

String[] names = {"Kian", "Maria", "Jose"};
```

Critical Rule — Arrays Are Zero-Indexed

The FIRST element is at index **0**, not 1. An array of 5 elements has valid indices 0, 1, 2, 3, 4 — there is NO index 5. This single fact causes more beginner bugs than almost anything else in this course.

Accessing & Modifying Elements

ARRAYDEMO.JAVA

```
public class ArrayDemo {
    public static void
main(String[] args) {
        int[] grades = {85, 90,
78, 92, 88};


System.out.println(grades[0]);

System.out.println(grades[2]);

System.out.println(grades.length);

        grades[1] = 95;

System.out.println(grades[1]);
    }
}
```

EXPECTED OUTPUT

```
85
78
5
95
```

LINE EXPANSION — EXECUTION TRACE

Index	0	1	2	3	4
Initial values	85	90	78	92	88
After grades[1] =95	85	**95**	78	92	88

`grades.length` gives `5` — the TOTAL count of elements, NOT the last valid index. The last valid index is always `length - 1`.

Gotcha Alert — ArrayIndexOutOfBoundsException

Trying to access `grades[5]` on a 5-element array crashes the program at runtime with `ArrayIndexOutOfBoundsException` — because valid indices only go up to `4` (which is `length - 1`). This is a RUNTIME error, not a compile error — the code compiles fine but crashes when actually executed.

Traversing Arrays with Loops

JAVA — STANDARD FOR LOOP

```
int[] grades = {85, 90, 78, 92,
88};
for (int i = 0; i <
grades.length; i++) {

System.out.println(grades[i]);
}
```

JAVA — ENHANCED FOR-EACH LOOP

```
int[] grades = {85, 90, 78, 92,
88};
for (int g : grades) {
    System.out.println(g);
}
```

Loop Style	Gives You Access To	Best When
Standard `for`	The INDEX (`i`) — can also modify elements via `grades[i] = ...`	You need the position, or need to modify values
Enhanced `for-each`	The VALUE directly, no index	Simple read-only traversal — cleaner, less error-prone syntax

Common Array Algorithms

Finding the Sum and Average

SUMAVG.JAVA

```
int[] scores = {85, 90, 78, 92,
88};
int sum = 0;
```

EXPECTED OUTPUT

```
Sum: 433
Average: 86.6
```

```
for (int s : scores) {
    sum += s;
}
double average = (double) sum /
scores.length;

System.out.println("Sum: " +
sum);
System.out.println("Average: " +
average);
```

Why (double) sum / scores.length, Not the Reverse

If you wrote `sum / scores.length` without casting, that's INT division (both are ints) — you'd get a truncated whole number like `86` instead of `86.6`. Casting `sum` to `(double)` FIRST forces the entire division to happen in decimal — this connects directly back to the integer division gotcha in Chapter 04.

Finding the Maximum Value

JAVA

```
int[] scores = {85, 90, 78, 92, 88};
int max = scores[0];  // assume first element is the max initially

for (int i = 1; i < scores.length; i++) {
    if (scores[i] > max) {
        max = scores[i];
    }
}
System.out.println("Max: " + max);  // Max: 92
```

2D Arrays — Arrays of Arrays

A 2D array represents a grid/table — rows and columns. Think of it as an array where each element is ITSELF another array. Just like 1D arrays, you can either declare-then-assign or use a literal:

JAVA — DECLARE-THEN-ASSIGN

```java
int[][] arr = new int[10][20];  // 10 rows, 20 columns, all zeros
arr[0][0] = 1;                   // row_index, then column_index
```

JAVA — ARRAY LITERAL

```java
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(grid[0][0]);  // 1  (row 0, col 0)
System.out.println(grid[1][2]);  // 6  (row 1, col 2)
System.out.println(grid[2][0]);  // 7  (row 2, col 0)

// Traversing a 2D array needs NESTED loops:
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        System.out.print(grid[row][col] + " ");
    }
    System.out.println();
}
```

Forward Link — Nested Loops (Chapter 06)

2D array traversal is EXACTLY the nested loop pattern from Chapter 06 — outer loop walks each row, inner loop walks each column within that row. If the Jollibee branch/register analogy made sense there, this is the same structure applied to grid data instead of physical locations.

CHAPTER 09

Introduction to Object-Oriented Programming

Everything up to this chapter has been **procedural** — a linear sequence of instructions. OOP is a different way of organizing code: modeling real-world things as **objects** that bundle together data (attributes) and behavior (methods). This chapter is intentionally light — full OOP mastery is the ENTIRE focus of **PF 101** next year. Here you just need the vocabulary and a working mental model.

Class vs Object — The Core Distinction

Concept	Definition
Class	A BLUEPRINT or template describing what an object WILL have (attributes) and WILL be able to do (methods). Does not exist as a real "thing" in memory by itself.
Object	A specific INSTANCE created FROM a class, with its own actual values. Multiple objects can be created from the same class, each with different data.

> **Filipino Analogy — Class vs Object**
> Think of the **Jollibee brand blueprint** as the class — it defines what every Jollibee branch WILL have (a menu, a location, operating hours) and WILL do (serve customers, process orders). Each actual physical Jollibee branch — SM North EDSA, Trinoma, Fairview — is an OBJECT: one specific instance built from that same blueprint, but each with its own actual address and staff.

Defining a Class

STUDENT.JAVA

```
public class Student {

    // Attributes (instance variables) — the DATA each object holds
    String name;
    int age;
    String course;

    // Constructor — special method that runs when an object is CREATED
```

```
    public Student(String n, int a, String c) {
        name = n;
        age = a;
        course = c;
    }

    // Method — BEHAVIOR the object can perform
    public void introduce() {
        System.out.println("Hi, I'm " + name + ", " + age +
            " years old, taking " + course + ".");
    }
}
```

Creating & Using Objects

MAIN.JAVA

```
public class Main {
    public static void
main(String[] args) {
        Student s1 = new
Student("Kian", 18, "BSIT");
        Student s2 = new
Student("Maria", 19, "BSCS");

        s1.introduce();
        s2.introduce();


System.out.println(s1.name);

System.out.println(s2.age);
    }
}
```

EXPECTED OUTPUT

```
Hi, I'm Kian, 18 years old,
taking BSIT.
Hi, I'm Maria, 19 years old,
taking BSCS.
Kian
19
```

LINE EXPANSION — EXECUTION TRACE

Step	What Happens
1	

Step	What Happens
	`new Student("Kian", 18, "BSIT")` — the `new` keyword allocates memory for a fresh object, then calls the constructor with these 3 arguments
2	Inside the constructor: `name="Kian"`, `age=18`, `course="BSIT"` get stored on THIS specific object, referenced by `s1`
3	`new Student("Maria", 19, "BSCS")` creates a COMPLETELY SEPARATE object, referenced by `s2` — s1 and s2 do not share data
4	`s1.introduce()` — dot notation calls the method ON that specific object, using s1's own name/age/course values
5	`s1.name` directly accesses that object's attribute — prints "Kian", confirming s1 and s2 are independent

Access Modifiers — Controlling Visibility

Modifier	Accessible From
`public`	ANYWHERE — any other class, any other file
`private`	ONLY within the same class — the standard way to protect an object's internal data (encapsulation)
`protected`	Same class, same package, AND subclasses (relevant once inheritance is introduced in PF 101)
(none / default)	Same package only

The Four Pillars of OOP — Preview Only

You will study these in full depth in **PF 101**. For now, just recognize the names and the one-sentence idea behind each.

Pillar	One-Sentence Idea
Encapsulation	Bundling data and methods together, hiding internal details behind `private` attributes and controlled `public` access
Inheritance	A class can inherit attributes/methods from a "parent" class, avoiding rewriting shared logic — same idea as the LGU → Barangay → Purok hierarchy from CC 101
Polymorphism	The same method name behaving differently depending on the object calling it — you already saw a taste of this with method overloading in Chapter 07
Abstraction	Hiding complex implementation details, exposing only what's necessary to use something (you use `System.out.println()` constantly without knowing HOW it actually prints to a screen)

> **Forward Link — PF 101 (Year 2, Semester 1)**
> This entire chapter is deliberately just the surface. **PF 101 — Object-Oriented Programming** requires BOTH **CC 103** and **PT 101** as prerequisites and is where classes, objects, constructors, and all four pillars above get their full, rigorous treatment — inheritance hierarchies, interface contracts, abstract classes, and polymorphic method overriding. Everything here is the seed; PF 101 is where it fully grows.